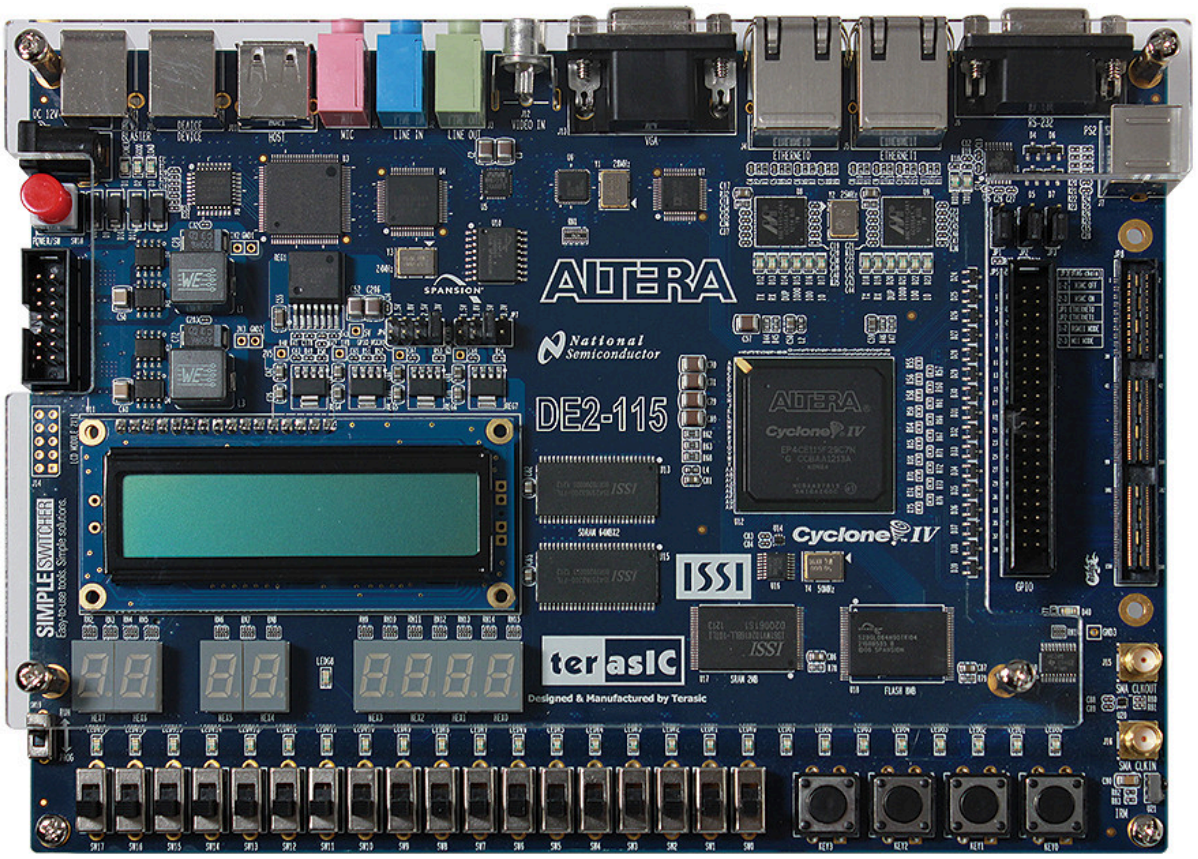
Memòria Pràctica 7



Estudiants:
Jordi Vivas Sakuta
Roger Balsebre Trias

Profesor laboratori:
Aarón Lozano Villanova

Index

1) Comptador 1
- Objectius
- Feina previa
- Realització al laboratori
- Resultats
- Conclusions

2) Comptador 2
- Objectius
- Descripció
- Realització al laboratori
- Resultats
- Conclusions

3) Comptador 3
- Objectius
- Descripció
- Realització al laboratori
- Resultats
- Conclusions

4) Conclusió general

1) Comptador 1

Objectius de la pràctica:

Aprofundir en el software ISE de Xilinx per a disseny VHDL per a dispositius programables. Utilitzar l'entorn Xilinx ISE per a la síntesi de circuits seqüencials. En concret s'aprendrà a: - Dissenyar un comptador, visualitzant els valors de sortida en els leds de la placa.

- Aprendre a fer descripcions en VHDL del comportament lògic del circuit a dissenyar.
- Utilitzar 'paquets' aritmètics de les llibreries VHDL estàndard.

Estudi previ

-Portar assimilats els conceptes adquirits a la pràctica 1 (repassar la Guia Xilinx ISE VHDL).
-Plantejar, a nivell de Màquina d'Estats Finits, el problema d'un comptador mòdul 8 (de 0 a 7), amb una sortida que indiqui si s'ha arribat al valor màxim. Modificar-lo per a que tingui entrades de reset i d'habilitació.

Realització al laboratori

- Counter

```
NET "Qout[0]" LOC = M5;
NET "Qout[1]" LOC = M11;
NET "Qout[2]" LOC = P7;
NET "Qout[3]" LOC = P6;
NET "Qout[4]" LOC = N5;
NET "Qout[5]" LOC = N4;
NET "rst" LOC = A7;
NET "Clk" LOC = G12;
NET "Clk" CLOCK_DEDICATED_ROUTE = FALSE;
```

Explicacions Counter

-Sortida programada per sis leds de la placa.
-Entrada del clock.

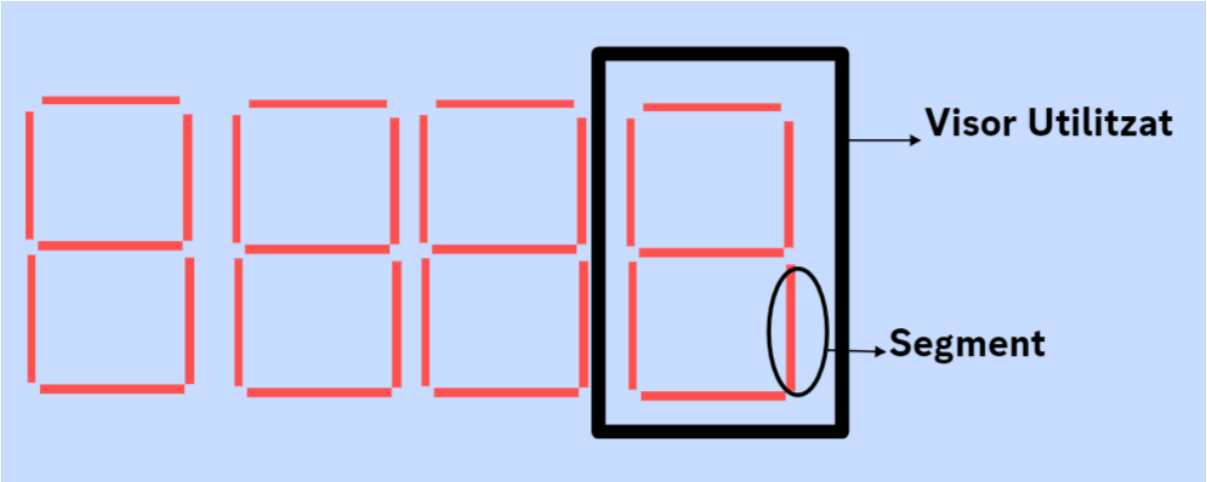
- Decoder codig

```
entity Decoder is
Port ( entrada : in STD_LOGIC_VECTOR (3 downto 0);
posicio : out STD_LOGIC_VECTOR (3 downto 0);
segment : out STD_LOGIC_VECTOR (6 downto 0);
punt : out STD_LOGIC);
end Decoder;
architecture Behavioral of Decoder is
begin
posicio <= "1110";
punt <= '1';
segment <= "0000001" when entrada= "0000" else
"1001111" when entrada= "0001" else
"0010010" when entrada= "0010" else
"0000110" when entrada= "0011" else
"1001100" when entrada= "0100" else
"0100100" when entrada= "0101" else
"1100000" when entrada= "0110" else
"0001111" when entrada= "0111" else
"0000000" when entrada= "1000" else
"0001100" when entrada= "1001" else
"1111111";
end Behavioral;
```

Explicació variables decoder

-Posició: és la poscició del visor què farem servir en aquest cas 1110 per tant el visor situat més a la dreta

-Segment: És cada un dels segments que formen el visor per tal de poder formar números visibles per nosaltre. El conjunt de tots els segments dóna la possibilitat de formar tots els números del 0 al 9.



Explicació codig Decoder

En el codig estem adaptant la entrada de números en binari a la sortida de cada segment corresponent per fer visible el propi número en deccimal en el visor.
És a dir si entra el 1 en binari (0001) la sortida serà (1001111) que correspon a la activació dels segments concrets de la dreta per tal de representar el número 1 en decimal.

- Decoder ucf

```
NET "posicio[0]" LOC = K14;
NET "posicio[1]" LOC = M13;
NET "posicio[2]" LOC = J12;
NET "posicio[3]" LOC = F12;
NET "segment[0]" LOC = M12;
NET "segment[1]" LOC = L13;
NET "segment[2]" LOC = P12;
NET "segment[3]" LOC = N11;
NET "segment[4]" LOC = N14;
NET "segment[5]" LOC = H12;
NET "segment[6]" LOC = L14;
NET "entrada[0]" LOC = P11;
NET "entrada[1]" LOC = L3;
NET "entrada[2]" LOC = K3;
NET "entrada[3]" LOC = B4;
NET "punt" LOC = N13;
```

Explicacions ufc

-Assignació de la variable posició als visors de la placa.
-Assignació de la variable segment a cada un dels segments corresponents del visor.

Resultats

Com a resultat hem obtingut un comptador binari amb un pulsador funcional el cual suma continuament amb això podem observar els números decimals en el visor.

Conclusions

Bona manera de fer-nos poder visualitzar els nostres projectes i poder preveure una utilitat d'aquests aprenentatge. Encara és un programa que es pot completar més com veurem més endavant.

2) Comptador 2

Objectius de la pràctica

- Aprofundir en el software ISE de Xilinx i el llenguatge VHDL per a disseny de circuits seqüencials:
- Fer servir el comptador de la sessió anterior com a base per a fer un divisor de freqüència.
- Aprofundir en les descripcions en VHDL del comportament lògic del circuit a dissenyar. - Aprendre a generar símbols i fer dissenys esquemàtics.

Descripció

A partir del circuit de la sessió anterior, dissenyar-ne un altre que anomenarem DivBy10 que compti de 0 a 9 i que tingui una entrada d'habilitació "E" i dues sortides de 1 bit: una ("Max") per a marcar el valor màxim (que com a l'apartat anterior sols val '1' al arribar al valor màxim i sols si el senyal "E" també val '1') i l'altra ("Clk_Out") que estigui a nivell alt la meitat dels cicles i a nivell baix l'altra meitat (sortida '0' si el valor comptat està entre 0 i 4 i sortida '1' si el valor està entre 5 i 9, i en aquest cas independent de l'entrada "E").

Realització al laboratori

- Divisor de freqüencia

divBy10

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- arithmetic functions with Signed or Unsigned values
use IEEE.NUMERIC_STD.ALL;
entity divBy10 is
	port(
		start, clk: in std_logic;
		Max, clk_out: out std_logic
	);
end divBy10;
architecture Behavioral of divBy10 is
signal actual, seguent, A: unsigned(3 downto 0);
```

```
signal B: std_logic;
begin
                    --parte combinacional
seguent<= actual when start ='0' else A;
        A<= "0000" when B='1' else actual +1;
        B <= '1' when actual = 9 else '0';
        max<= B and start;
        clk_out <= '1' when actual > 4 else '0';

        --Flip-flops
        -- ...
        process(clk)
        begin
         if(clk'event and clk='1') then
                  actual <= seguent;
                 else
                  actual<=actual;
                 end if;
end process;
end Behavioral;
```

divBy50K

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- arithmetic functions with Signed or Unsigned values
use IEEE.NUMERIC_STD.ALL;
entity divBy50k is
	port(
		start, clk: in std_logic;
		Max, clk_out: out std_logic
	);
end divBy50k;
architecture Behavioral of divBy50k is
signal actual, seguent, A: unsigned(15 downto 0);
signal B: std_logic;
begin
	--parte combinacional
seguent<= actual when start ='0' else A;
	A<=x"0000" when B='1' else actual +1;
	B <= '1' when actual = 49999 else '0';
	max<= B and start;
	clk_out <= '1' when actual > 24999 else '0';

	--Flip-flops
	-- ...
	process(clk)
	begin
	 if(clk'event and clk='1') then
```

```
                    actual <= seguent;
                   else
                    actual<=actual;
                   end if;
end process;
end Behavioral;
```

Explicació divisor de freqüencies

Aquest codi és un divisor de freqüència. És com un comptador que agafa el rellotge d'entrada (`clk`) i el fa més lent. Això passa perquè compta fins a 50.000 i després es reinicia.
Per aconseguir la freqüencia que volen hem de fer els següents divisors:

1. **Primer divisor per 50.000 (50k):**
    - El divisor de 50k redueix la freqüència inicial de 50 MHz a 1.000 Hz. Això significa que ara tens un pols cada mil·lisegon (1 ms).
    - **Codi del primer divisor:** Ja el tens definit amb el codi de `divBy50k`.
2. **Segon divisor per 10:**
    - Amb el senyal sortida de 1.000 Hz del primer divisor, ara necessites un segon divisor que divideixi per 10. Això et donarà un senyal de 100 Hz.
3. **Tercer divisor per 10 (un altre):**
    - Finalment, afegeixes un altre divisor per 10. Això agafarà el senyal de 100 Hz i el reduirà a 10 Hz, i amb un quart divisor per 10 arribaràs a 1 Hz.

Resultat

Com a resultat hem obtingut un programa capaç de sumar-se unitats per cada segon el que vindria a ser un rellotge incomplet.

Conclusió

És una extenció de la primera part, però com he mencionat anteriorment encara està incomplet, tot i que el projecte ja està agafant forma i ja va encaminat cap a un objectiu marcat.

3) Comptador 3

Objectius de la pràctica

- Aprofundir en el software ISE de Xilinx i el llenguatge VHDL per a disseny de circuits seqüencials:

- Fer servir el comptador de segons de la sessió anterior com a base per a fer un cronòmetre / rellotge.

- Aprofundir en les descripcions en VHDL del comportament lògic del circuit a dissenyar. - Aprendre a generar símbols i fer dissenys esquemàtics.

Descripció

La sessió consisteix en encadenar comptadors i afegir el control necessari per a passar d'un cronòmetre de segons a un rellotge de 24 hores (un rellotge és un cronòmetre de 24 hores amb opcions per a sincronitzar l'hora amb una referència externa). En una 1a fase s'afegiran sols els minuts, però després es tracta de tenir també hores i de poder veure el valor de hores i minuts en 4 dígits 7 segments. Com a suport es proporciona al fitxer auxiliar "Project_P7C.zip" que incorpora el circuit de visualització. Si el feu servir com a base, i completeu els codis igual que heu fet al projecte anterior visualitzareu els segons als 7 segments. Una vegada comprovat el funcionament, serà el moment de practicar en la realització de dissenys "esquemàtics", connectant entre si les diferents entitats prèviament dissenyades i convertides a símbols. Es demana que modifiqueu el disseny, creant i afegint els blocs que calgui, per a tenir un cronòmetre de 24 hores (visualitzant hores i minuts als visors i els segons als leds. Per a acabar, modifiqueu el disseny dels SSS de minuts i hores per a que incorporin 2 botons per a modificar l'hora del rellotge (un per a incrementar minuts i l'altre per a incrementar les hores) En aquests 2 apartats finals heu de ser autosuficients, i cal que com a memòria feu un petit manual del procediment.

Realització al laboratori

Aquesta pràctica va consistir en aprofitar les dues anteriors pràctiques per tal de fer un cronòmetre automatic l'organització va ser la següent:

1. **Divisor de freqüència:**
    - Redueix la freqüència d'entrada fins a **1 Hz** (divisor principal de segons).
2. **Comptadors encadenats:**
    - **Comptador de segons (0-59):** Afegeix un comptador que compta de 0 a 59 amb sortida a minuts.
    - **Comptador de minuts (0-59):** Connectat al comptador de segons.
    - **Comptador d'hores (0-23):** Connectat al comptador de minuts.
3. **Visualització:**
    - Mostra segons als LEDs.
    - Mostra minuts i hores als 7 segments utilitzant el codi `Decoder`.
4. **Control manual:**
    - Implementa botons per ajustar els minuts i hores del rellotge.

Resultat

El disseny del **Comptador 3** encadena mòduls de comptadors i divisors de freqüència per crear un rellotge complet de 24 hores. Amb els botons d'ajust manual, podem modificar l'hora, i la visualització en LEDs i dígits de 7 segments completa el projecte.

Conclusió

La finalització i la completació de les pràctiques ha sigut aquesta última necessitant de tots el components anteriors per tal de acabar-la, ja és un projecte definitiu i complet on es mostra el coneixement aprés en la asignatura.

4) Conclusió general

Aquestes pràctiques ens han permès entendre i aplicar els conceptes de disseny de circuits seqüencials amb VHDL, treballant amb comptadors, divisors de freqüència i visors. Hem integrat els aprenentatges per desenvolupar un rellotge funcional, consolidant la teoria i millorant les nostres habilitats en disseny digital i ús d'eines professionals com Xilinx ISE.